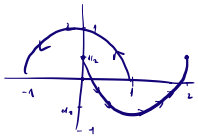
Implementación del código

$N = 400$ datos y ruido $\sigma = 0.12$ con StandardScaler

Generamos las lunas sin ruido en el plano: $[0,\pi] \longrightarrow \mathbb{R}^2$, $\theta \longmapsto (\cos\theta, \sin\theta)$ y $[0,\pi] \longrightarrow \mathbb{R}^2$, $\theta \longmapsto (1-\cos\theta, 1-\sin\theta-0.5)$



Añadimos ruido a cada punto p: $p = p_{perfecto} + \text{random_normal}(0, 0.12)$

Aplicamos $p_{final} = \frac{p-\mu}{\sigma}$ para que tengan media 0 y dt 1. (Standard Scaler)

$\gamma_0 = \frac{1}{N}\sum_{i=1}^{N} \delta_{(x_0^i, y_0^i)} \in P(\mathbb{R}^{d_1} \times \mathbb{R}^{d_2})$ es la medida de datos inicial.

<u>Arquitectura</u> Tenemos $X_0 \in \mathbb{R}^2$ y $F(x,t) = \frac{1}{M}\sum_{m=1}^{M} \underbrace{\tanh(a_1^m \cdot x + a_t^m \cdot t + b^m)}_{\in \mathbb{R}} \cdot \underbrace{a_0^m}_{\in \mathbb{R}^2} \in \mathbb{R}^2$

$\nu_t = \frac{1}{M}\sum_{m=1}^{M} \delta_{a^m}$

$F(x,t) = \int_A b(x,a)\, d\nu_t(a) \approx \int_A b(x,a) \frac{1}{M}\sum_{m=1}^{M} \delta_{a^m}\, da = \underbrace{W_0}_{2\times M} \underbrace{\tanh(W_1 (x,t)^T + b)}_{M\times 1}$ con $W_0 = \frac{1}{M}\begin{pmatrix} a_{0,1}^1 & a_{0,1}^2 & \dots & a_{0,1}^M \\ a_{0,2}^1 & a_{0,2}^2 & \dots & a_{0,2}^M \end{pmatrix}_{2\times M}$,

$= \frac{1}{M}\sum_{m=1}^{M} b(x, a^m)$

$W_1 = \begin{pmatrix} a_{1,1}^1 & a_{1,2}^1 & a^1 \\ a_{1,1}^2 & a_{1,2}^2 & a^2 \\ \vdots & & \\ a_{1,1}^M & a_{1,2}^M & a^M \end{pmatrix}_{M\times 3}$, $b = \begin{pmatrix} b^1 \\ \vdots \\ b^M \end{pmatrix}_{M\times 1}$.

Aquí parámetros fijos en $t \in [0,T]$ y la dep. temporal viene con t explícito. (diferencia con el paper)

Obtenemos X_T con RK4: $X_{t+dt} = X_t + \frac{dt}{6}(K_1 + 2K_2 + 2K_3 + K_4)$

Tras obtener $X_T \in \mathbb{R}^2$, predecimos que $P(y=1|x) = \sigma(\text{logit}) = \sigma(W_c \cdot X_T + b_c)$ con $W \in \mathbb{R}^2$, $b \in \mathbb{R}$ $\quad \sigma(x) = \frac{1}{1+e^{-x}} \begin{cases} 1, & x \gg 0 \\ 1/2, & x \approx 0 \\ 0, & x \ll 0 \end{cases}$

<u>Entrenamiento</u> $J = BCE_N + \varepsilon \frac{1}{N_p}\sum_j (0.05\,\theta_j^4 + 0.5\,\theta_j^2)$ con el optimizador Adam + cosine annealing + + clipping del gradiente (max_norm = 5)

$$BCE_N = -\frac{1}{N}\sum_{i=1}^{N} [y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$$

N_p: nº de parámetros en MeanFieldVelocity; en W_1, W_0 y b (llamado W_1 bias). W_c y b_c

$W_1 \in \mathbb{R}^{M\times(d_1+1) = 64\times 3}$
$W_0 \in \mathbb{R}^{d_1 \times M = 2\times 64}$
$b \in \mathbb{R}^{M=64}$, $W_c \in \mathbb{R}^2$, $b_c \in \mathbb{R}$
$\Rightarrow N_p = 6\cdot 64 + 3 = 387$

(Dividimos para que el valor no dependa del tamaño del modelo)

Tenemos que $\nu_t = \frac{1}{M}\sum_{m=1}^{M} \delta_{a^m}$ y $\nu^\infty \propto e^{-U(a)}$. Sabemos que $KL(\nu_t|\nu^\infty) = \begin{cases} \int_A \log \frac{d\nu_t}{d\nu^\infty}\, da & \text{si } \nu_t \ll \nu^\infty \\ +\infty & \text{si } \nu_t \not\ll \nu^\infty \end{cases}$

Tenemos que $\nu_t \not\ll \nu^\infty$ ya que sea $A_j = \{a_j\}$ para algún $j \in \{1,\dots,M\}$ $\Rightarrow \nu^\infty(A_j) = 0$ al ser medida continua pero $\nu_t(A_j) = \frac{1}{M} \neq 0$. En conclusión, $\nu_t \not\ll \nu^\infty$, $KL(\nu_t|\nu^\infty) = +\infty$.

Sin embargo, el término de energía sí es computable y es:

$$E_{\nu_t}[\ell(a)] = \int_A \ell(a)\, d\nu_t(a) \approx \frac{1}{M}\sum_{m=1}^{M} \ell(a^m) = \frac{1}{M}\sum_{m=1}^{M}\left(c_1 |a^m|^4 + c_2 |a^m|^2\right)$$

donde a^m es con la notación de arriba $a^m = (a^m_{1,1}, a^m_{1,2}, a^m_t, a^m_{0,1}, a^m_{0,2}, b^m) \in \mathbb{R}^6$, y $|a^m|^2 = \sum_{k=1}^{6}(a^m_k)^2$ y $|a^m|^4 = \left(|a^m|^2\right)^2$. (el código calcula $\sum_{k=1}^{6}\left((a^m_k)^2\right)^2$ en vez de $\left(\sum_{k=1}^{6}(a^m_k)^2\right)^2$)

<u>Adam</u> Para cada parámetro θ_j, $m_0 = 0$, $m_{t+1} = \beta_1 m_t + (1-\beta_1)\nabla_{\theta_j} J$ con $\beta_1 = 0{,}9$, $\beta_2 = 0.999$
$v_0 = 0$, $v_{t+1} = \beta_2 v_t + (1-\beta_2)(\nabla_{\theta_j} J)^2$

Así, $\theta_{j+1} = \theta_j + \eta_t \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$ con $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$

<u>Cosine annealing</u> $\eta_0 = 0.005 = \eta_{max}$, $\eta_t = \dfrac{\eta_{max}}{2}\left(1 + \cos\left(\dfrac{\pi t}{T_{max}}\right)\right) \xrightarrow[t \to T_{max}]{} 0$

<u>Gradient clipping</u> Para evitar gradientes explosivos en las primeras fases del entrenamiento, y por tanto la actualización rápida de los parámetros en Adam, se escala el vector $\nabla J = \left(\dfrac{\partial J}{\partial \theta_1}, \dots, \dfrac{\partial J}{\partial \theta_{N_p}}\right)$ si su norma supera un umbral. En concreto,

$$\text{Si } |\nabla J| = \sqrt{\sum_{j=1}^{N_p}\left(\frac{\partial J}{\partial \theta_j}\right)^2} > 5 \Rightarrow \frac{\partial J}{\partial \theta_j} \leftarrow \frac{\partial J}{\partial \theta_j}\,\frac{5}{|\nabla J|}$$

<u>Back-propagation</u> $\dfrac{\partial J}{\partial \theta_j} = \dfrac{\partial J}{\partial \text{logit}}\,\dfrac{\partial \text{logit}}{\partial X_T}\,\dfrac{\partial X_T}{\partial \theta_j}$ esto lo computa PyTorch con loss.backward()